

Pembuatan Middleware Komunikasi Menggunakan XMPP dan Arsitektur Microservice

Resa Fajar Sukma^{1, a)}, Muhammad Eka Suryana^{2, b)} Med Irzal^{3, c)}

¹*Program Studi Ilmu Komputer Fakultas Matematika dan Ilmu Pengetahuan Alam.*

²*Universitas Negeri Jakarta.*

³*Jalan Raya Rawamangun Muka, Jakarta Timur DKI Jakarta 13220.*

Email: ^{a)}rezasukma96@gmail.com, ^{b)}eka-suryana@unj.ac.id, ^{c)}medirzal@unj.ac.id

Abstract

Developing an application requires time, effort, and resources. One common feature found in applications is communication. This research aims to reduce the time developers spend developing communication features by utilizing existing middleware. This middleware uses XMPP as the data exchange protocol, XMPP employs long-lived TCP connection socket programming, and the middleware utilizes a microservice architecture to divide large functionalities. Functionalities are divided into three main parts: addressing each component represented by the tracker component, the managerial process represented by the manager component, and message delivery represented by the relay component. The primary objective of this research is to transform the monolithic XMPP architecture into a microservice architecture, which will empower each major functionality to become a best practice for the specific cases faced by each functionality. The results of this research demonstrate that the transformation from monolithic to microservice provides flexibility in developing and implementing each component without the fear of damaging other components, and yields positive results in the ability to apply best practices for each case faced by the component.

Keywords: middleware, XMPP, socket programming, microservice.

Abstrak

Aplikasi membutuhkan waktu, tenaga, dan biaya untuk mengembangkannya. Salah satu fitur umum yang terdapat pada aplikasi adalah komunikasi. Penelitian ini dibuat untuk memangkas waktu pengembang aplikasi dalam mengembangkan fitur komunikasi, dengan cara menggunakan middleware yang sudah dapat digunakan. Middleware ini menggunakan XMPP sebagai protokol pertukaran data, XMPP menggunakan socket programming tipe long-lived TCP connection, dan middleware menggunakan arsitektur microservice untuk membagi setiap fungsionalitas besarnya. Fungsionalitas dibagi menjadi tiga bagian besar yaitu pengalamatan setiap komponen yang diwakili oleh komponen tracker, proses manajerial yang diwakili oleh komponen manager, dan pengiriman perpesanan yang diwakili oleh komponen relay. Tujuan utama dari penelitian ini adalah mengubah arsitektur monolith XMPP menjadi microservice yang akan memberikan kemampuan setiap fungsionalitas besar menjadi best practice terhadap kasus yang dihadapi oleh masing-masing fungsionalitas. Hasil dari penelitian ini menunjukkan perubahan dari monolith ke microservice, memberikan keleluasan dalam pengembangan tiap komponen dan penerapannya, tanpa takut merusak komponen lainnya serta memberikan hasil positif berupa kemampuan menerapkan best practice dari setiap kasus yang dihadapi komponen.

Kata-kata kunci: middleware, XMPP, socket programming, microservice.

PENDAHULUAN

Komunikasi di zaman sekarang dapat dilakukan di mana saja dan kapan saja, tidak memedulikan jarak dan waktu pengiriman selama memiliki jaringan internet. Komunikasi dilakukan melalui gawai atau komputer setiap orang, namun sayangnya komunikasi tidak dapat terjadi jika perangkat memiliki jaringan yang berbeda. Cara menanganinya adalah dengan aplikasi.

Aplikasi membutuhkan waktu untuk pembuatannya, mulai dari membuat tampilan depan dan juga alur pengiriman data yang dilakukan oleh aplikasi tersebut. Aplikasi juga memiliki banyak fitur atau kebutuhan sesuai dengan tujuannya masing-masing, sehingga untuk mengembangkan fitur komunikasi dalam aplikasi pasti membutuhkan waktu tersendiri untuk memikirkan alur pengiriman data maupun tampilannya. Saat mengembangkan aplikasi, pengembang juga memiliki pilihan untuk menggunakan aplikasi penghubung atau middleware.

Middleware adalah aplikasi penghubung yang dikembangkan untuk tujuan tertentu, contohnya komunikasi. Middleware membantu pengembang dalam mempercepat pengembangan aplikasinya karena pengembang hanya perlu memikirkan tampilan, mengirimkan data ke middleware, dan menerima data dari middleware. Hal ini mengartikan bahwa middleware akan mengurus permasalahan alur pengiriman data agar sesuai dengan tujuannya.

Alur pengiriman data untuk tujuan komunikasi secara real-time memiliki sebuah protokol yang disebut XMPP. XMPP adalah teknologi open source untuk komunikasi secara real-time dan memiliki berbagai kegunaan. XMPP tercatat dalam dokumentasi Internet Engineering Task Force (IETF) pada RFC 6120 untuk teknologi inti dan RFC 6121 untuk Instant Messaging and Presence.

RFC 6120 membahas mengenai teknologi inti seperti arsitektur dari XMPP. XMPP memiliki arsitektur monolith, format data XML, dan tipe koneksi yang digunakan adalah long-lived TCP connections (P. Saint-Andre, 2011). Arsitektur monolith adalah sistem yang dikembangkan menjadi satu aplikasi utuh, jika aplikasi dideploy pada server dan server mati maka layanan juga akan terhenti seluruhnya. Cara mengatasinya adalah menggunakan arsitektur microservice yaitu layanan dipisahkan menjadi bagian kecil sesuai dengan bagiannya. (Chandler Harris, 2024). Format data XML adalah format data yang tidak efisien jika dibandingkan dengan format data JSON (Breje et al., 2018). Koneksi tcp akan memakan sumber daya server, maka jika tipenya long-lived TCP connections dan banyak yang melakukan koneksi ke server akan mengakibatkan server menjadi lambat atau berhenti (Maxwin, 2021).

Penggunaan XMPP sebagai jalur penghubung komunikasi dalam aplikasi sudah sering diterapkan. Contohnya pembuatan aplikasi web instant messaging (Gupta et al., 2021).

KAJIAN PUSTAKA

XMPP

Extensible Messaging Presence And Protocol (XMPP) adalah teknologi terbuka untuk komunikasi secara real-time, menggunakan Extensible Markup Language (XML) sebagai format dasar untuk pertukaran informasi. XMPP menyediakan cara untuk melakukan pengiriman potongan-potongan kecil XML dari satu entitas ke entitas lain dalam waktu yang hampir mendekati instan.

XMPP digunakan untuk jangkauan variasi aplikasi yang beragam. Pembahasan lebih dalam mengenai variasi penggunaan XMPP dapat dibagi menjadi dua hal yaitu dari sisi services dan dari sisi aplikasi. Services dalam XMPP terbagi menjadi dua bagian yaitu yang utama dan extension. Pembahasan mengenai services yang utama dipublikasikan oleh Internet Engineering Task Force (IETF) pada situs <http://ietf.org> disebut sebagai seri RFC (Request For Comments), dan mengenai service berupa extension dipublikasikan oleh XMPP Standards Foundation (XSF) pada situs <http://xmpp.org> disebut sebagai seri XEP (XMPP Extensible Protocol). Aplikasi adalah program perangkat lunak yang disebarkan melalui skenario ketertarikan umum secara individu ataupun organisasi (Saint-Andre et al., 2009).

XMPP menggunakan arsitektur client-server dengan addressing setiap client-nya seperti addressing pada email yaitu `user@dns.com`. Komunikasi pada XMPP bertipe long-lived TCP connections dan

pertukaran data dalam XMPP dilakukan dalam bentuk stanza. Stanza adalah unit standar komunikasi dalam XMPP.

Middleware

Middleware adalah komponen perangkat lunak yang menjembatani aplikasi, data, dan pengguna. Membantu menyederhanakan pengembangan aplikasi dan membuat pengembang bekerja secara lebih efisien karena mereka dapat berfokus pada pengembangan fitur perangkat lunak yang diinginkan (Revou, 2024).

Microservice

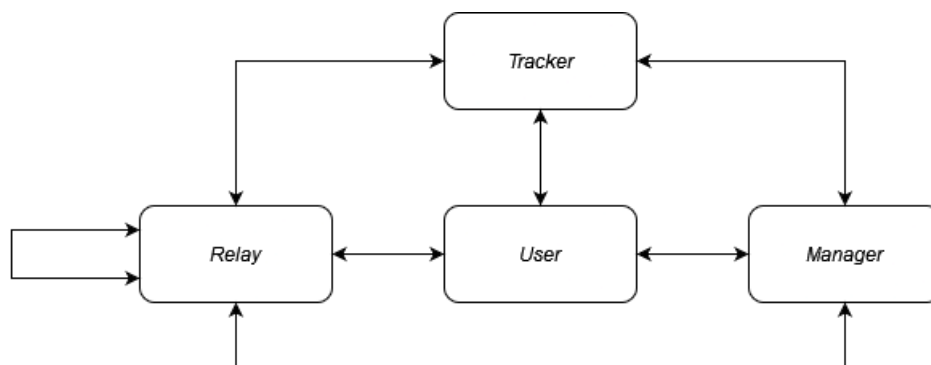
Arsitektur microservice adalah model arsitektur yang mengandalkan serangkaian layanan yang dapat diterapkan secara independen. Setiap layanan memiliki logika bisnis dan database tersendiri untuk mencapai tujuan layanannya. Kelebihan arsitektur ini adalah kemudahan dalam menskalakan aplikasi, kemudahan dalam maintenance aplikasi dan melakukan pengetestan, penerapan yang mandiri, fleksibilitas teknologi yang akan digunakan untuk layanan tertentu, ketika memulai ulang suatu layanan tidak akan mempengaruhi layanan yang lainnya. Kelemahan arsitektur ini adalah jika tidak ada yang mengepalai pengembangan maka pengembangan aplikasi akan menjadi lambat, harga yang harus dibayarkan untuk infrastruktur semakin besar, debugging menjadi lebih kompleks (Chandler Harris, 2024).

Relay

Relay adalah sistem telekonferensi, yang sangat cocok untuk dijalankan di banyak mesin secara terdistribusi. Pengaturan yang dipakai adalah sebuah server menjadi titik keterhubungan client, melakukan pengiriman pesan atau fungsi yang lainnya (Oikarinen & Reed, 1993).

METODE PENELITIAN

Arsitektur Sistem



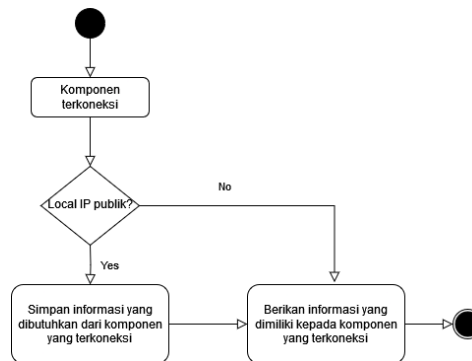
GAMBAR 1. Arsitektur Sistem

Arsitektur akan dibuat sebagai microservice, dengan setiap service akan dibedakan berdasarkan komponen utama. Arsitektur akan memiliki tiga komponen utama yaitu tracker, manager, dan relay dan satu komponen penguji yaitu user. Adapun peran dari masing-masing komponen adalah sebagai berikut:

1. Tracker: Memiliki peran sebagai tempat pengelola informasi IP yang dimiliki oleh masing-masing komponen.
2. Manager: Memiliki fungsi untuk manajerial seperti pengaturan keseimbangan jaringan pada relay, pengelolaan aktivitas user seperti autentikasi, kontak, dan status.
3. Relay: Memiliki peran untuk menjembatani data yang berasal dari satu user ke user yang lain.
4. User: Memiliki peran sebagai klien dari jaringan yang telah dibuat oleh tracker, manager, dan relay.

Arsitektur akan dibangun dengan jaringan terkoneksi menggunakan socket bertipe TCP (Transmission Control Protocol) long-lived connections. Sistem socket tipe TCP menggunakan teknik koneksi (connection) antara dua socket client. Format protokol untuk socket TCP adalah (socket_protocol, client_1_address, client_1_port, client_2_address, client_2_port).

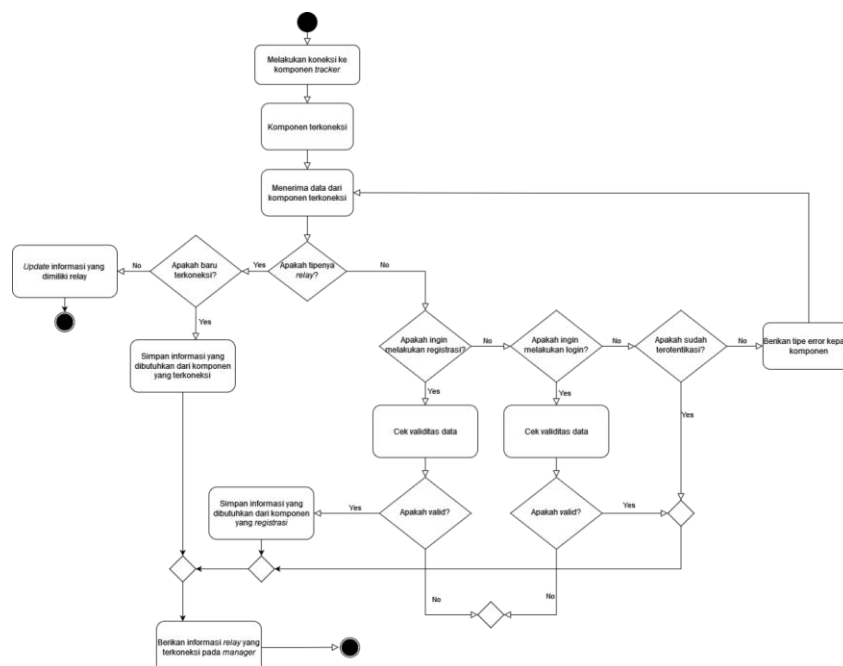
Flowchart Komponen Tracker



GAMBAR 2. Flowchart Komponen Tracker

Tracker akan menjadi komponen yang pertama dinyalakan pada sistem middleware, karena tugas dari tracker adalah mengelola alamat komponen manager dan relay. Komponen manager dan relay, setelah tracker dinyalakan akan melakukan koneksi kepada tracker dan memberikan data diri kepada tracker.

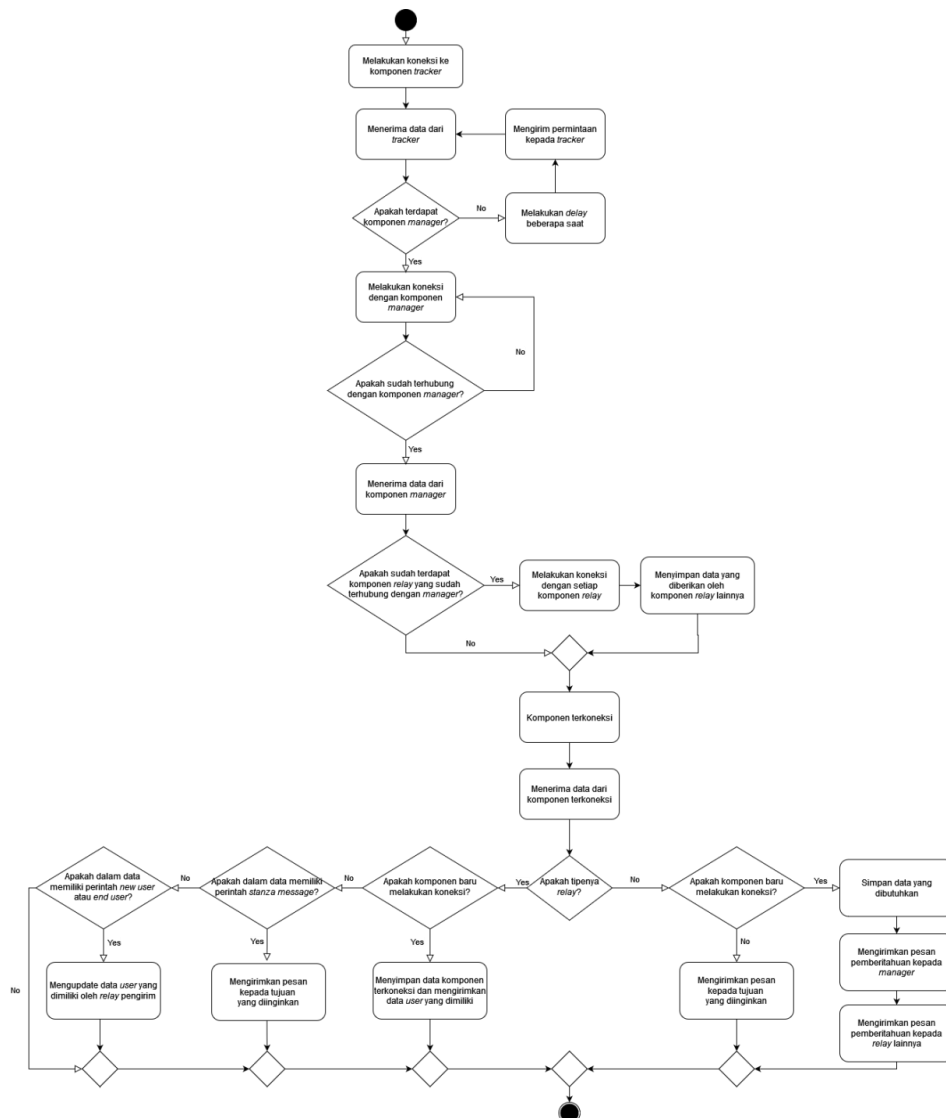
Flowchart Komponen Manager



GAMBAR 3. Flowchart Komponen Manager

Manager ketika dijalankan akan melakukan koneksi kepada komponen tracker dan memberikan data dirinya. Kemudian manager akan menjadi sebuah server yang siap menerima koneksi dari komponen relay ataupun user. Manager berfungsi sebagai load balancer koneksi relay, dan pengelolaan kebutuhan user seperti roster, dan presence.

Flowchart Komponen Relay



GAMBAR 4. Flowchart Komponen Relay

Relay ketika dinyalakan akan melakukan koneksi kepada komponen tracker dan memberikan data dirinya. Kemudian melakukan koneksi kepada manager, memberikan data dirinya, dan melakukan konfigurasi awal, jika sudah ada relay lain yang terkoneksi pada manager. Setelah itu, relay akan menjadi server dan siap menerima koneksi dari user.

Format Stanza XMPP JSON

Ada tiga jenis stanza dalam XMPP, yaitu message, presence, dan iq (info/query). Pengubahan dari masing-masing stanza dari XML menjadi JSON akan terlihat seperti gambar dibawah.

```
{
  "stanza": "message",
  "from": "user_initiator",
  "to": "user_target",
  "lang": "id",
  "body": "value_of_message",
  "time_send": "timestamp_now"
}
```

GAMBAR 5. Stanza Message JSON

```
{
  "stanza": "presence"
}
```

GAMBAR 6. Stanza Presence JSON

```
{
  "stanza": "iq",
  "namespace": "roster",
  "from": "user_initiator",
  "type": "set",
  "query": {
    "item": {
      "jid": "user_target",
      "name": "alias_name_for_user_target",
      "subscription": "to"
    }
  }
}
```

GAMBAR 7. Stanza IQ Namespace Roster JSON

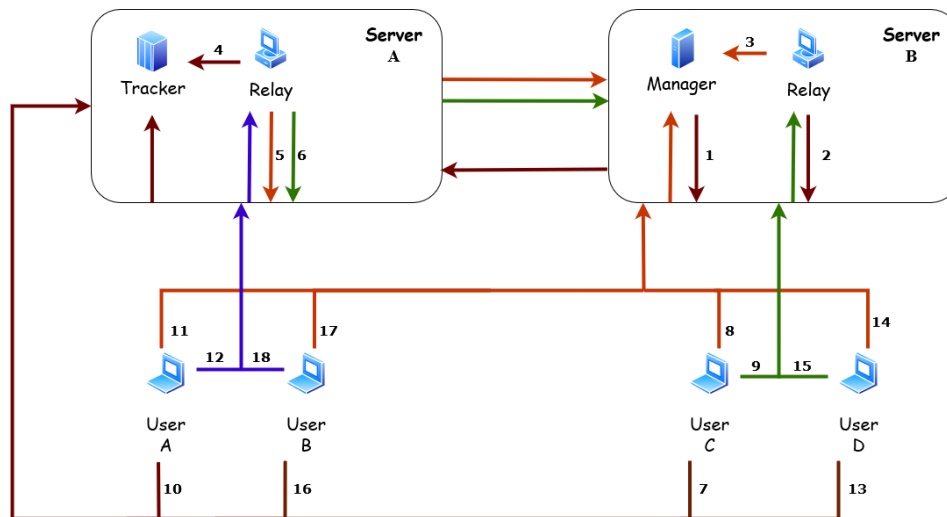
Alat Penelitian

Adapun alat yang digunakan dalam penelitian ini antara lain adalah:

1. VSCode.
2. SQLite3.
3. Python 3.
4. VPS.
5. Perangkat komputer/laptop pribadi.

HASIL DAN PEMBAHASAN

Implementasi Sistem Middleware



GAMBAR 8. Implementasi Sistem Middleware

Sistem akan dijalankan pada dua server berbeda, dimana pada server A diterapkan komponen tracker dan relay, dan pada server B diterapkan komponen manager dan relay. Menjalankan sistem dalam dua server adalah untuk menguji apakah sistem dapat melakukan peningkatan kemampuan sesuai kebutuhan, yang mana dalam hal ini adalah kemampuan menambahkan komponen relay sebagai jalur komunikasi antar user dalam sistem. Keterangan dari GAMBAR 8 adalah sebagai berikut:

1. Panah yang keluar dari suatu komponen menandakan bahwa komponen tersebut akan melakukan koneksi kepada komponen lainnya.
2. Komponen target koneksi diwakili dengan warna, sebagai berikut:
 - Coklat, mewakili komponen target tracker.
 - Jingga, mewakili komponen target manager.
 - Hijau, mewakili komponen target relay pada server B.
 - Biru, mewakili komponen target relay pada server A.
3. Urutan angka yang berada disekeliling komponen, menandakan urutan mereka akan melakukan koneksi, dimulai dari angka terkecil hingga terbesar.

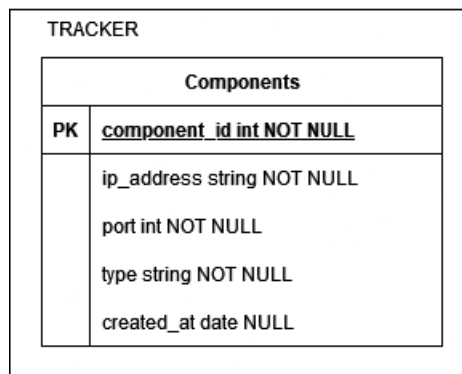
Cara menjalankan sistem yang telah dibuat adalah, menyalakan komponen tracker, kemudian manager, dan terakhir relay. Setelah ketiganya dinyalakan, sistem siap untuk membantu user saling berkomunikasi.

manager	2/23/2025 1:16 AM	File folder
relay	2/23/2025 12:58 AM	File folder
tracker	2/23/2025 12:57 AM	File folder
utils	9/19/2024 11:14 PM	File folder
user	10/11/2024 2:32 PM	File folder
user bot joker node JS	2/10/2025 2:08 PM	File folder

GAMBAR 9. Struktur Direktori

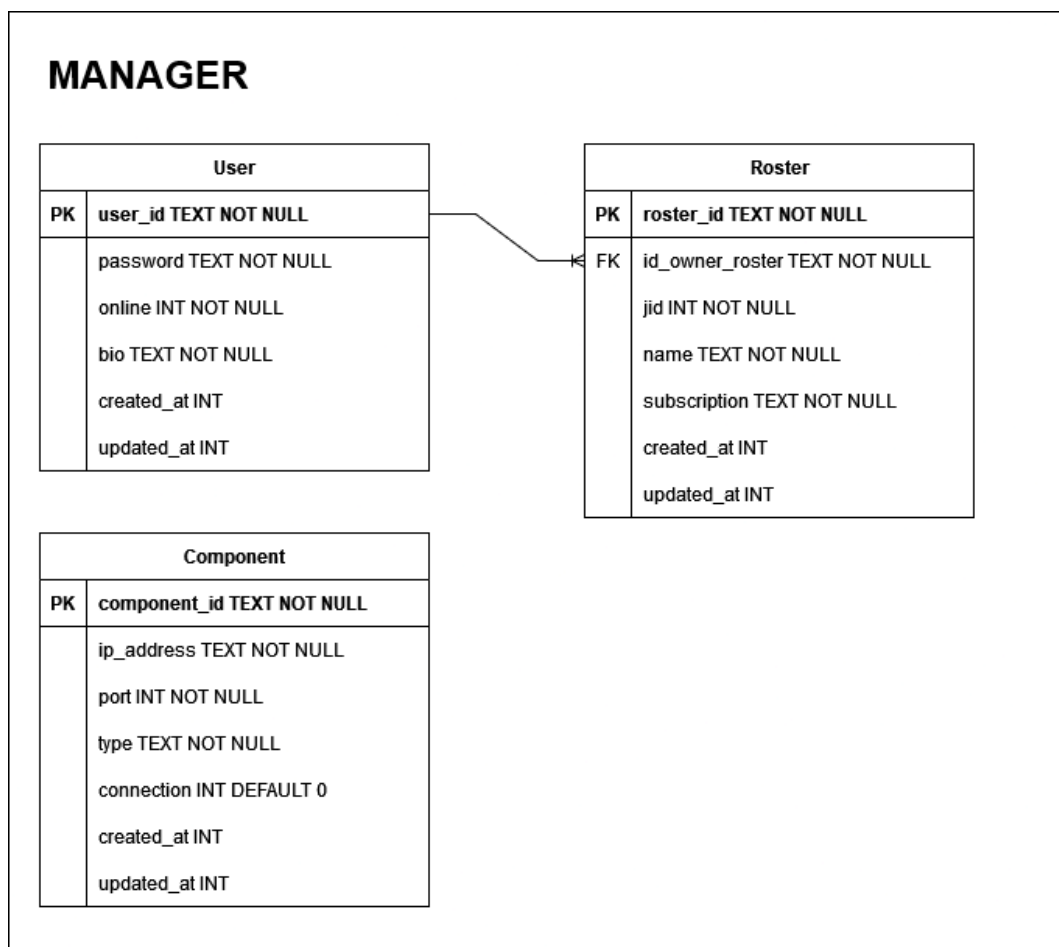
Terdapat lima folder berbeda, tiga diantaranya adalah folder komponen utama yaitu, manager, relay, dan tracker, satu komponen fungsionalitas yaitu utils, dan dua komponen pengujian yaitu user dan user bot joker nodeJS. Untuk kelengkapan kode dapat dilihat pada github <https://github.com/Jarsters/socket-xmpp-json/>.

Implementasi Database Middleware



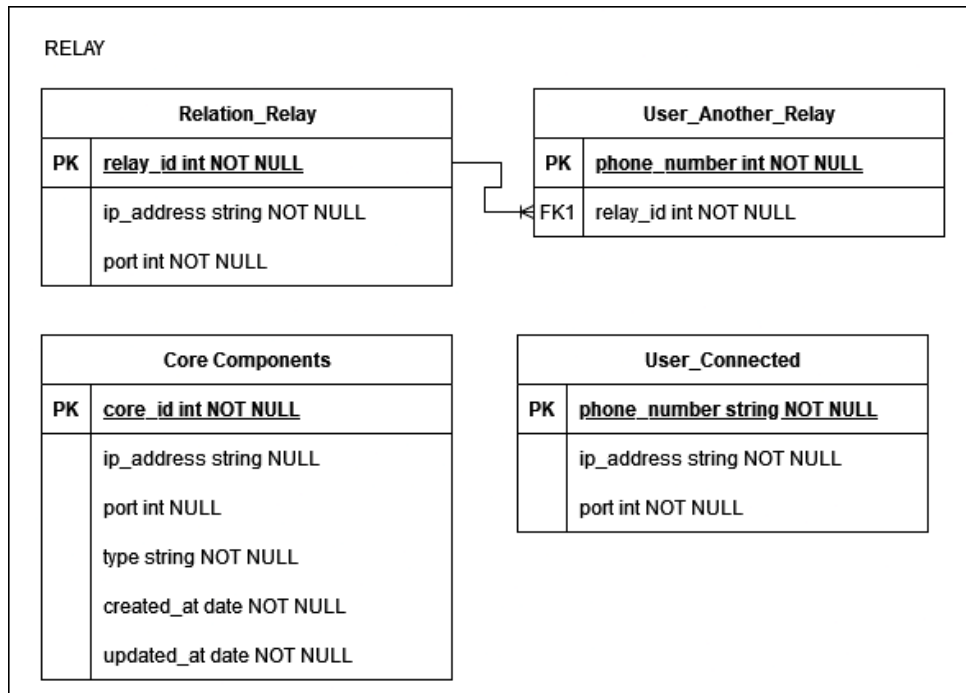
GAMBAR 10. Database Tracker

Tracker membutuhkan sebuah penyimpanan data untuk mengelola data komponen manager dan relay.



GAMBAR 11. Database Manager

Manager membutuhkan penyimpanan data untuk mengelola pengelolaan load balancer koneksi untuk relay, pengelolaan data diri user, daftar kontak user (roster), dan presence. Tabel component untuk pengelolaan relay, dan tabel user dan roster untuk kebutuhan pengelolaan user.



GAMBAR 12. Database Relay

Relay membutuhkan tempat penyimpanan data untuk mengelola pesan yang diterima olehnya akan dikirimkan ke user siapa dan berada dimana. Tabel relation_relay adalah sebuah tabel yang menyimpan data tentang relay lain yang terhubung dalam sistem middleware dan user_another_relay adalah tabel pengalamatan user siapa berada di-relay yang mana. Tapi sebelum mencari seorang user berada dimana sistem akan mencari apakah user yang ditargetkan terkoneksi dengan dirinya, hal ini diwakili oleh tabel user_connected yang mana menyimpan data user yang terkoneksi dengan dirinya.

Pengujian Sistem

Sistem yang dibuat diuji dengan cara pengujian unit testing, yaitu pengujian fungsionalitas dari masing-masing komponen, apakah sudah sesuai seperti saat sistem masih berarsitektur monolith atau belum. Adapun hasil pengujian fungsionalitasnya dapat dilihat pada tabel dibawah.

TABEL 1. Daftar Pengujian Fungsionalitas Komponen Tracker

Skenario Pengujian	Yang Diharapkan	Pengamatan	Kesimpulan
Menerima koneksi dari komponen lainnya	Tracker akan mampu menerima koneksi, membedakan, mencatat alamat komponen lain, dan mengubah status mereka dari aktif menjadi tidak aktif	Tracker mampu menerima koneksi, membedakan, mencatat alamat komponen lain, dan mengubah status mereka dari aktif menjadi tidak aktif	Lolos pengujian.
Menerima packet pesan dari komponen lainnya	Tracker akan mampu menerima dan mengelola packet pesan yang diterima dengan baik dan benar.	Tracker mampu menerima dan mengelola packet pesan agar sesuai dengan kebutuhan dari packet pesan yang diterima.	Lolos pengujian.

TABEL 2. Daftar Pengujian Fungsionalitas Komponen Manager

Skenario Pengujian	Yang Diharapkan	Pengamatan	Kesimpulan
Menjadi klien komponen tracker	Manager akan melakukan koneksi ke tracker dan memberikan data dirinya kepada komponen tracker	Manager mampu melakukan koneksi dan memberikan data dirinya ke komponen tracker.	Lolos pengujian.
Menjadi server dan menerima koneksi dari komponen relay dan user	Manager akan mampu menerima koneksi dari komponen relay dan user	Manager mampu menerima koneksi dari komponen relay dan user	Lolos pengujian.
User melakukan autentikasi pada sistem	Manager akan mampu membantu user untuk melakukan registrasi ataupun login	Manager mampu membantu user untuk melakukan registrasi ataupun login	Lolos pengujian
Melakukan load balancer untuk koneksi relay	Manager akan mampu memberikan relay dengan koneksi paling sedikit kepada user	Manager mampu memberikan relay dengan koneksi paling sedikit kepada user	Lolos pengujian
User melakukan pengelolaan terhadap rosternya seperti melakukan CRUD.	Manager akan mampu membantu user untuk mengelola CRUD roster yang dimiliki user tersebut.	Manager mampu membantu user untuk mengelola CRUD roster yang dimiliki user tersebut.	Lolos pengujian
User melakukan inialisasi presence	Manager akan mampu membantu user untuk memberitahukan kepada yang lainnya bahwa dirinya sedang online	Manager mampu membantu user untuk memberitahukan kepada yang lainnya bahwa dirinya sedang online	Lolos pengujian
User melakukan permintaan directed presence	Manager akan mampu memberikan presence user lainnya kepada user yang meminta, tanpa user tersebut harus melakukan subscribe kepada user target	Manager mampu memberikan presence user lainnya kepada user yang meminta, tanpa user tersebut harus melakukan subscribe kepada user target	Lolos pengujian
User melakukan pembaruan bio yang dimilikinya	Manager akan mampu memperbarui bio user, dan memberitahukan kepada subscriernya bahwa user tersebut telah melakukan pengubahan bio	Manager mampu memperbarui bio user, dan memberitahukan kepada subscriernya bahwa user tersebut telah melakukan pengubahan bio	Lolos pengujian
User logout dari sistem	Manager akan mampu memperbarui status dari user tersebut, dan memberitahukan kepada subscriernya bahwa user tersebut telah offline	Manager mampu memperbarui status dari user tersebut, dan memberitahukan kepada subscriernya bahwa user tersebut telah offline	Lolos pengujian

TABEL 3. Daftar Pengujian Fungsionalitas Komponen Relay

Skenario Pengujian	Yang Diharapkan	Pengamatan	Kesimpulan
Menjadi klien komponen tracker dan manager	Relay akan melakukan koneksi ke tracker dan memberikan data dirinya kepada tracker, kemudian melakukan koneksi kepada manager dan memberikan data diri kepada manager	Relay melakukan koneksi ke tracker dan memberikan data dirinya kepada tracker, kemudian melakukan koneksi kepada manager dan memberikan data diri kepada manager	Lolos pengujian.
Menjadi server dan menerima koneksi dari komponen relay dan user	Relay akan mampu menerima koneksi dari komponen relay dan user	Relay mampu menerima koneksi dari komponen relay dan user	Lolos pengujian.
Melakukan pengelolaan informasi user yang melakukan koneksi	Relay akan mampu, saat ada user yang terkoneksi, relay akan menyimpan data user, menginformasikan kepada manager bahwa koneksinya bertambah, dan memberikan data user terkoneksi kepada relay lainnya yang berada pada sistem agar dicatat oleh relay lainnya	Relay mampu, saat ada user yang terkoneksi, relay akan menyimpan data user, menginformasikan kepada manager bahwa koneksinya bertambah, dan memberikan data user terkoneksi kepada relay lainnya yang berada pada sistem agar dicatat oleh relay lainnya	Lolos pengujian
Melakukan pengelolaan informasi user yang melakukan diskoneksi	Relay akan mampu, saat ada user yang terdiskoneksi, relay akan menghapus data user, menginformasikan kepada manager bahwa koneksinya berkurang, dan memberikan data user yang terdiskoneksi kepada relay lainnya yang berada pada sistem agar dihapus dari catatan oleh relay lainnya	Relay mampu, saat ada user yang terdiskoneksi, relay akan menghapus data user, menginformasikan kepada manager bahwa koneksinya berkurang, dan memberikan data user yang terdiskoneksi kepada relay lainnya yang berada pada sistem agar dihapus dari catatan oleh relay lainnya	Lolos pengujian
Menerima pesan dari relay atau user	Relay akan mampu menerima dan mengelola pesan yang diterima dari relay dan user	Relay mampu menerima dan mengelola pesan yang diterima dari relay dan user	Lolos pengujian
Meneruskan pesan kepada user atau relay lainnya	Relay akan mampu meneruskan pesan kepada target yang sesuai	Relay mampu meneruskan pesan kepada target yang sesuai	Lolos pengujian

KESIMPULAN DAN SARAN

Kesimpulan

Berdasarkan implementasi dan pengujian untuk middleware komunikasi menggunakan XMPP dan arsitektur microservice ini, dapat disimpulkan bahwa:

1. Socket programming hanyalah saluran untuk pertukaran data. Dibutuhkan sebuah protokol pertukaran data untuk menangani saluran ini seperti XMPP.
2. Pemahaman tentang protokol yang digunakan sebagai landasan format pertukaran data harus dikuasai dengan benar. Pemahaman ini akan membantu jika pengembang ingin melakukan inovasi pada protokol tersebut.
3. Pembuatan middleware komunikasi menggunakan XMPP dan arsitektur microservice mempunyai tujuan khusus untuk menambahkan kemampuan meningkatkan kebutuhan yang dibutuhkan saja. Hal ini dapat dilihat dari pengimplementasian sistem, komponen yang berfungsi sebagai jalur komunikasi yaitu relay dapat digandakan. Tidak seperti monolith yang tidak memiliki kemampuan meningkatkan kebutuhan tertentu, monolith ketika ingin meningkatkan kebutuhan, wajib meningkatkan seluruh proses.
4. Pengubahan dari monolith ke microservice memberikan kemampuan untuk memberikan hasil yang terbaik dari setiap komponen. Hasil ini didapat karena setiap komponen dapat dikembangkan sesuai best practice kasus yang dihadapi, tanpa takut mengganggu proses komponen lain.

Saran

Adapun saran untuk penelitian selanjutnya adalah:

1. Keamanan jaringan pada penelitian ini belum diimplementasikan sama sekali, maka ada baiknya jika memperbaiki permasalahan keamanan jaringan yang ada. Mengamankan jaringan dengan algoritma enkripsi ECC dan mengamankan pengiriman data yang dilakukan menggunakan enkripsi AES.
2. Pada penelitian ini, fitur konfirmasi penambahan roster masih otomatis dilakukan oleh server. Bisa menambahkan spesifikasi lebih lanjut pada fitur ini.
3. Fitur untuk mengirimkan gambar atau pesan suara juga bisa ditambahkan.

UCAPAN TERIMA KASIH

Terima kasih kepada keluarga tercinta, dosen yang membantu dalam pembuatan skripsi, dan teman-teman yang selalu memberi dukungan.

REFERENSI

- Breje, A. R., Gyorödi, R., Gyorödi, C., Zmaranda, D., & Pecherle, G. (2018). Comparative study of data sending methods for XML and JSON models. *International Journal of Advanced Computer Science and Applications*, 9(12), 198–204. <https://doi.org/10.14569/IJACSA.2018.091229>.
- Chandler Harris. (2024). Microservices vs. monolithic architecture. <https://www.atlassian.com/microservices/microservices-architecture/microservices-vs-monolith>.
- Gupta, D., Shivankar, J., & Gugulothu, S. (2021). Instant messaging using xmpp. *Journal of Physics: Conference Series*, 1913(1), 8–12. <https://doi.org/10.1088/1742-6596/1913/1/012126>.
- Maxwin, A. (2021). TCP/IP concurrent connections. <https://learn.microsoft.com/en-us/answers/questions/482793/tcp-ip-concurrent-connections>.
- Oikarinen, J., & Reed, D. (1993). Internet Relay Chat Protocol. Internet Engineering Task Force, May, 1–65. <https://www.rfc-editor.org/info/rfc1459>.
- P. Saint-Andre. (2011). Extensible Messaging and Presence Protocol (XMPP): Core. <https://xmpp.org/rfcs/rfc6120.html>.
- Revou. (2024). Middleware. PT Revolusi Cita Edukasi. <https://revou.co/kosakata/middleware>.
- Saint-Andre, P., Smith, K., & Tronçon, R. (2009). XMPP : The Definitive Guide.